Supplementary Material for "Super-Precise Latency: How CPU Threading Affects High-Precision Latency, and an Industry-Wide Audit"

Gustavo Pedro Ricou and Romaric Duvignau

This document is the supplementary material for the paper named above, which points to the sections here that hold its evidence. Timestamps and spans are those of Equation 1 and Fig. 1 of the paper, and both processes read one clock unless a result says otherwise. Table S1 defines the terms used here that the paper does not. The complete record of how the results were obtained, with every result that did not survive, is a separate document in the archived artifact, the postmortem. Pointers such as "postmortem, S37" refer to its sections.

S1. The setup in full

What a reader needs to repeat a run is here: the testbeds and their clocks, every setting that can move a latency number, and where each timestamp is taken.

S1.1. Testbeds, clocks and workload

Two testbeds carry the evidence (Table S2). On the cloud testbed the producer and the consumer run as separate processes on one host with 8 online CPUs, and read one wall clock, CLOCK_REALTIME, through `time.time_ns()`. A wall clock gives every host one epoch, which the cross-host runs need, and on one host its readings came out in order: no latency timed from the publish call falls below zero (S1.3). No scheduler tunable is written anywhere in the archived campaign, which is strong evidence, short of proof, that the kernel's defaults applied, among them the derived base slice of 2.8 ms (Section IV-C). The clocksource was not recorded, but a measured read cost narrows down which one it was (S3). Across hosts, `chrony` reports an inter-host offset of 0.067 ms but bounds its own error at 4–7 ms per host over the 4 hosts captured, far more than the offset, so we claim no cross-host bound.

The producer replays StatsBomb's open football event data [S1]. On the cloud testbed it replays eleven of its $3{,}315$ matches, in order, at a chosen publish rate. The registered campaign replays a steady 50 messages a second, except in its block M0, which replays a match's bursts.

S1.2. Every setting that can move a latency number

Table S3 states every client setting that can move a latency number, and why it holds its value.

G. P. Ricou is with the School of Computer Science and Statistics, Trinity College Dublin, Dublin 2, Ireland. E-mail: pedrorig@tcd.ie.
R. Duvignau is with the Department of Computer Science and Engineering, Chalmers University of Technology, Gothenburg, Sweden. E-mail: duvignau@chalmers.se.

TABLE S1
Terms used here that the paper does not define.

Term	Meaning
cell	A condition together with a workload; where the workload is fixed, the two coincide. A run executes a cell once, and a replicate is a further run of it.
configuration	One setting of a manipulation that compares settings differing in one variable.
core count k	The number of cores the background load occupies.
grid	The instants, τ apart, at which a timestamp changes value.
q-lattice	The retentions $0, 1/q, \ldots, 1$ a paced producer can reach when its publish interval over τ is a/q in lowest terms.

TABLE S2
EACH TESTBED CARRIES A DIFFERENT PART OF THE EVIDENCE. MACHINES, SOFTWARE, AND WHAT RESTS ON EACH TESTBED.

Testbed	Machines	Kernel and software	What rests on it
Cloud	Four Oracle Cloud `VM.Standard.E5.Flex` instances on a private network: three brokers and one driver.	Ubuntu 22.04, kernel `6.8.0-1057-oracle` (EEVDF scheduler), a 1 ms tick; CPython 3.10; `chrony`-disciplined clocks, logged every minute.	Every measurement the paper does not attribute to the registered campaign or its machines.
Registered campaign	Three Microsoft Azure pairs in one region, each a driver with eight virtual CPUs and a broker with two: two x86 pairs (AMD EPYC 9V74) and one Arm pair (Neoverse N2).	The provider's stock kernels, 6.8.0-1064 and 6.8.0-1065; for the tick block, three kernels built from one source tree that differ in the tick alone, 1, 4 and 10 ms.	The registered campaign (S6), with its tools block, and the later runs that time the receiving thread's wait.

TABLE S3
EVERY CLIENT SETTING THAT CAN MOVE A LATENCY NUMBER, AND WHY IT HOLDS ITS VALUE.

Setting	Value	Why
Kafka producer		
`acks`	`all`	Durability matched to Redis's synchronous `XADD`.
`linger.ms`	0	No artificial batching delay: on a sparse workload any linger would dominate the quantity measured.
`max-inflight`	64	At our producer's default of 1 the client blocks on a round trip per message.
`compression`	none	Football events are small JSON. Compression would add CPU work to the timed path for no representative benefit.
`ack-stamp`	`callback`	The default, asymmetric with Redis (S1.3).
Redis producer		
publish workers	pool	Non-blocking dispatch keeps the publish loop on schedule; publish and acknowledgment are timestamped in the worker.
Consumers		
Redis `ack-batch`	200	Acknowledging each message separately adds a round trip per message.
Redis `count`	200	Read batch, matched to the acknowledgment batch so neither dominates.
Kafka `poll-timeout`	1000 ms	The client already amortizes fetches. No per-record round trip exists to remove.
Replay		
speed-up	derived	Plans carry a built-in $120\times$ compression, so the flag is not the rate. The rate is derived from the plan and checked against wall time before every campaign.

S1.3. Where each timestamp is taken, and which spans fall below zero

The publish timestamp is read on the calling thread immediately before the client is invoked. The two clients read the acknowledgment timestamp in different places, but in both it is read only after a thread that was not running has been woken and then given a core. Under Redis it is read on the calling thread the moment the synchronous `xadd` returns, with no second thread involved. Under Kafka it is read, by default, in the client's I/O thread when the delivery callback fires. Our Kafka producer's `inline` mode instead timestamps on the calling thread once the publish future resolves, and requires one request outstanding.

The consumer reads t_{recv} when it holds the record, and t_{out} once it has handled it. Our Kafka consumer parses the payload inside `poll()`, before t_{recv}, and our Redis consumer parses it between the two. The processing-time latency $t_{\mathrm{out}} - t_{\mathrm{recv}}$ is therefore 290 ns under Kafka and 20,772 ns under Redis, medians of run medians over 5,863 runs, and negative on none of the 708,505 messages. The parse thus falls inside D for Kafka only, and Redis's median end-to-end latency of $800\,\mu$s leaves out a parse 2.6% as long. A longer D puts fewer values of S below zero, and Kafka already has the higher rate, so including that parse would widen the gap between the brokers in Table I.

Table I counts the values below zero in each span over every cloud run that produced matched records, rejected runs included, because a rejected run shows which span falls below zero as well as a clean one does. It leaves out the messages a consumer received late, while it was still reading earlier runs' messages (S1.5). Only S, the one span whose two timestamps have no causal order (Section IV-A), ever does.

S1.4. Which runs each number rests on

The paper's numbers rest on eight bodies of runs, each chosen by its own rule (Table S4). They overlap but do not coincide, so a count from one is not a count from another.

TABLE S4
EACH NUMBER RESTS ON A STATED BODY OF RUNS. WHERE A NUMBER COMES FROM, HOW ITS RUNS WERE CHOSEN, AND WHERE THE PAPER AND THIS SUPPLEMENT USE IT.

Numbers	Runs and messages	How the runs were chosen	Where
The distribution of S, 8.79% below zero; the message size and publish rate, from the same runs' producer logs	5,863 runs, 708,505 messages	every cloud run with matched records, rejected or not	II, III; S1.3
The sign check	884 runs in 40 conditions	the runs behind a reported result	VI; S2
The understatement, the recovery and the exposure band	70 groups from 51 conditions, 665,299 messages	each condition's runs, split by the sign check's verdict	VI; S2.1
The manipulations	25 runs and 2,408–2,985 messages a configuration, on each broker	the campaigns E-A3 to E-A10	IV-B; S3
The traced waits	551,956 wakeups	E-A9, normal priority, 88% load	IV-C; S3.7
The positivity filter	75 settings with a captured summary	the benchmark, embedded mode	V-C; S4
The public reports	43 reports	the registered audit's search and rule	V-D; S5
The registered campaign	8,102 runs in 156 campaigns, and 1,516,342 acknowledgments recorded	three Azure pairs, a plan frozen before its runs	IV, VIII; S6

S1.5. Two faults of the cloud testbed, and the messages left out

Two faults of the cloud testbed's consumers came to light after its runs, when the testbed was gone and no run could be repeated. One lost messages. The other delayed them, and it decides which messages every number from that testbed counts.

The consumer stopped during long pauses: A consumer stops after 15 s without a message. The producer replays a match in real time, and many matches pause for longer than that. In 1,491 runs the consumer stopped during such a pause, and the producer published the rest of the match to a broker that acknowledged it and that no one read: 91,177 of the 829,907 messages published, each among its run's last. A message no one received has no end-to-end latency, so no number counts these.

The consumer first read earlier runs' messages: The runs reused topic and stream names from one campaign to the next, and nothing cleared them. The Redis trial deleted its stream with a command-line client the driver did not have, and discarded the error; the Kafka trial cleared nothing. A run's consumer reads its topic or stream from the beginning and discards other runs' messages. So it first read every earlier campaign's, a median of 6,010 messages on Redis, whose consumer logs each read, and meanwhile the run's own first messages waited in the broker. A message that waited there was received long after its acknowledgment. It cannot put S below zero, so it dilutes the rate.

The rule: A message is late when its delivery took more than a second longer than the run's fastest. The consumer caught up when it received the last such message. A message published before then that arrived within a second of that instant came in the same burst, so it is late too, although it waited less. Every message of a run is late when even its fastest delivery took over a second, because its consumer never caught up. The second is a wide margin. A run's fastest delivery took 1.5 ms at the median and under 111 ms in 99% of runs, and over a second only in the 50 runs whose consumer never caught up.

How the rule was checked: None of the 30,225 messages it finds has a value of S below zero. In every one of the 2,099 runs that have any, the run's first published message is one of them, so each backlog began with its run. Every Redis run with a late message whose consumer logged its reads had read other runs' messages first. The late messages are 2.2% of Kafka's matched messages and 6.0% of Redis's.

What leaving them out changes: Every analysis of the cloud testbed leaves the late messages out (`scripts/stale_backlog.py`). The corpus of Table I falls from 738,730 matched messages to 708,505, and its negative-span rate rises from 8.43% to 8.79%. The campaigns whose consumers read the longest backlogs move most. The padding repeat E-A10b had sat below E-A10 and now agrees with it at every padding (Table S8). The traced campaigns' rates rise to their untraced twin's, which removes a finding: the tracer had seemed to lower Kafka's rate at normal priority, and it does not (S3.5). The cells of a table now hold different numbers of messages, and each caption gives the range.

Shorter waits: A backlog the consumer read in under a second escapes the rule, which needs a wait of a second to see one. Leaving out also each run's leading messages whose S exceeds the run's median by more than 300 ms, with those published before the last of them arrived, takes out 1,898 more messages, none below zero. The corpus rate then reads 8.81%, so the margin's choice moves it by hundredths of a point.

What cannot be recomputed: The sign check's counts in S2 come from runs that are no longer in the archive, and were made with the late messages in. A late message can only dilute its run's values below zero, so the check's rejections are a lower bound. The registered campaign deleted its streams and topics before each run. By the same rule 0.06% of its messages are late, and only 31 of its 9,055 runs began with a wait, so its readings stand as judged (S6).

S2. THE SIGN CHECK, AND THE RUNS IT COVERS

The rule is that of Section VI, applied to every run. A run is rejected if more than 1% of its messages carry a negative value in S, in the publish delay $t_{\mathrm{pub}} - t_{\mathrm{sched}}$ or in the processing-time latency $t_{\mathrm{out}} - t_{\mathrm{recv}}$, or if the median of any of the three is

TABLE S5
SECTION VI'S NUMBERS COME FROM 70 GROUPS OF RUNS. GROUP: BROKER (K, R), SENDER COUNT, FEED COUNT, AND THE SIGN CHECK'S VERDICT ON THE GROUP'S RUNS (P PASSED, F REJECTED). D, A AND S: MEDIANS, IN MILLISECONDS. A/D: THE ERROR OF S AS A LATENCY, IN PERCENT. REPAIR: THE ERROR OF THE RECOVERY THAT ADDS THE MEDIAN OF A TO THE MEDIAN OF S, IN PERCENT OF THE MEDIAN OF D. GENERATED AT BUILD TIME FROM THE COMMITTED TABLE.

Group	D	A	S	A/D	Repair	Group	D	A	S	A/D	Repair
K 12 1 p	2.05	1.50	0.50	73	2	R 12 12 f	0.70	0.50	0.10	71	29
K 12 10 f	2.00	1.45	0.55	72	0	R 12 12 p	0.70	0.50	0.10	71	29
K 12 10 p	1.95	1.45	0.50	74	0	R 12 3 f	0.80	0.50	0.10	62	12
K 12 12 p	1.95	1.45	0.50	74	0	R 12 4 f	0.85	0.50	0.10	59	6
K 12 3 p	2.00	1.45	0.50	72	0	R 12 5 f	0.70	0.50	0.10	71	29
K 12 4 p	2.00	1.45	0.55	72	0	R 12 5 p	0.70	0.50	0.10	71	29
K 12 5 p	2.05	1.50	0.50	73	0	R 12 6 p	0.75	0.50	0.10	67	20
K 12 6 p	2.00	1.45	0.50	72	2	R 12 7 f	1.10	0.50	0.60	45	18
K 12 7 p	2.00	1.45	0.50	72	0	R 12 8 f	1.05	0.50	0.20	48	14
K 12 8 f	2.00	1.45	0.50	72	0	R 12 9 f	0.75	0.50	0.10	67	20
K 12 9 p	2.00	1.50	0.50	75	0	R 12 9 p	0.70	0.50	0.05	71	29
K 1 1 p	2.15	1.65	0.45	77	0	R 1 1 f	0.70	0.55	0.05	79	36
K 2 1 p	1.75	1.30	0.35	74	6	R 1 1 p	0.70	0.55	0.05	79	36
K 2 2 p	1.60	1.20	0.35	75	3	R 2 1 f	0.80	0.55	0.10	69	19
K 5 1 f	2.85	2.20	0.70	77	0	R 2 1 p	0.75	0.50	0.10	67	20
K 5 1 p	2.65	1.90	0.65	72	2	R 5 1 f	1.20	0.65	0.20	54	8
K 5 2 f	2.95	2.25	0.70	76	0	R 5 1 p	1.20	0.60	0.15	50	17
K 5 2 p	2.65	1.85	0.65	70	4	R 5 2 f	1.20	0.65	0.25	54	8
K 5 3 f	2.85	2.15	0.70	75	0	R 5 2 p	1.20	0.65	0.15	54	12
K 5 3 p	2.60	1.85	0.65	71	2	R 5 3 f	1.20	0.65	0.40	54	8
K 5 4 f	2.90	2.20	0.70	76	0	R 5 3 p	1.15	0.65	0.15	57	30
K 5 4 p	2.70	1.90	0.65	70	4	R 5 4 f	1.25	0.65	0.40	52	12
K 5 5 f	2.75	2.05	0.60	75	4	R 5 4 p	1.30	0.65	0.65	50	19
K 5 5 p	2.80	1.95	0.85	70	0	R 5 5 f	1.15	0.65	0.20	57	4
K 9 1 p	2.05	1.50	0.50	73	2	R 5 5 p	1.20	0.65	0.10	54	38
K 9 3 p	2.00	1.45	0.50	72	0	R 9 1 f	0.80	0.55	0.10	69	19
K 9 4 p	2.10	1.50	0.55	71	2	R 9 1 p	0.75	0.50	0.10	67	20
K 9 5 p	2.05	1.50	0.50	73	0	R 9 3 p	0.70	0.50	0.10	71	29
K 9 6 p	1.95	1.45	0.50	74	0	R 9 4 f	0.80	0.50	0.10	62	12
K 9 7 p	2.00	1.45	0.50	72	0	R 9 5 p	0.70	0.50	0.10	71	29
K 9 8 f	1.95	1.45	0.50	74	3	R 9 6 p	0.75	0.50	0.10	67	20
K 9 9 p	1.95	1.45	0.50	74	0	R 9 7 f	1.10	0.50	0.60	45	18
R 12 1 f	0.80	0.55	0.10	69	19	R 9 8 p	1.05	0.50	0.50	48	14
R 12 10 f	1.05	0.50	0.55	48	14	R 9 9 f	0.70	0.50	0.10	71	29
R 12 10 p	1.00	0.50	0.50	50	10	R 9 9 p	0.70	0.50	0.05	71	29

negative. A condition is usable only if all of its runs survive. The one-percent line is a convention, and we fixed the rule after earlier campaigns had run, which is weaker than registration.

Over the 884 cloud runs behind our results, the check rejects 459 (51.9%), and 13 of their 40 conditions keep every run. These counts were made with the late messages of S1.5 in, from runs no longer in the archive, so leaving those messages out could only add rejections. That corpus is not the one of Table I, which counts every cloud run that produced matched records, rejected or not, 5,863 of them. Of those, 3,985 exceed the one-percent line on S.

S2.1. The groups of runs behind the paper's Section VI

Section VI's understatement, the recovery's errors and the exposure band are computed over 70 groups of runs, 665,299 messages, one group for each condition and sign-check verdict, from 51 conditions (Table S5). For each group the table gives the medians of D, A and S, the error of S as a latency, A/D, and the error of the recovery that adds the median of A to the median of S, as a share of the median of D. The understatement is one over the median, across the groups, of the median of S over the median of D, and its interquartile range is over the same groups. The recovery's median errors are over the groups the sign check passes and over those it rejects. The shares of messages whose publish latency exceeds half or all of the end-to-end latency are counted message by message over the corpus of Table I.

S3. THE MECHANISM AND ITS LAWS IN FULL

Codes beginning E-A name campaigns on the Oracle Cloud testbed, and a code ending in b repeats or extends the one before it. Only the receiving thread's wait (S3.8) was measured elsewhere, on the registered campaign's machines.

S3.1. Real-time priority cuts the rate in every matched pair

Moving the threads that read the clock to real-time priority, `SCHED_FIFO`, cut the negative-span rate between 7- and 80-fold on Kafka, and between 6.0- and 7.8-fold on Redis, in each of eight matched pairs, with utilization unchanged (Table S6). At real-time priority those threads preempt the background load instead of queueing behind it, so their waiting probability falls while the machine stays as busy. Any account in which utilization alone sets the rate predicts no change.

The pairs come from three campaigns. E-A5 is the original manipulation, E-A5b widened the range of loads to 60 and 95%, and E-A7 replicated E-A5. A rule fixed in advance withholds a pair whose two achieved utilizations differ by more than 0.05. It withheld 0 pairs, and the widest difference in the table, 0.002, is 25 times smaller than the tolerance.

TABLE S6
FACTOR OF 7 TO 80, WITH UTILIZATION MATCHED IN EVERY PAIR. NORMAL-PRIORITY AND REAL-TIME NEGATIVE-SPAN RATES AT THE SAME ACHIEVED UTILIZATION, AND THE FACTOR BETWEEN THEM, FROM THREE CAMPAIGNS, ON KAFKA; THE PAPER'S SECTION IV-B GIVES REDIS'S. EVERY CELL IS 2,973 TO 2,985 MATCHED EVENTS. BOTH CONFIGURATIONS' ACHIEVED ρ IS PRINTED SO THAT THE MANIPULATION CAN BE CHECKED PAIR BY PAIR. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES.

campaign	load	ρ normal	ρ real-time	normal	real-time	factor
E-A5b	60%	0.6055	0.6035	0.0459	0.0017	27×
E-A5b	70%	0.7032	0.7025	0.1032	0.0144	7×
E-A5	75%	0.7531	0.7525	0.1320	0.0034	39×
E-A7	75%	0.7550	0.7537	0.1504	0.0044	35×
E-A5b	88%	0.8806	0.8809	0.2223	0.0034	66×
E-A5	88%	0.8809	0.8812	0.3049	0.0057	54×
E-A7	88%	0.8825	0.8825	0.2532	0.0054	47×
E-A5b	95%	0.9501	0.9501	0.2943	0.0037	80×

Only the normal-priority rate moves with load. The real-time rate stays between 0.0017 and 0.0144 over the whole ladder, as it should if priority moves the waiting probability p of Equation 3 directly. The factor tends to grow with load but not at every step, so it is reported as a range. For E-A5's two pairs its Katz 95% intervals [S2] are [21–74] at 75% load ($z = 19.8$) and [33–86] at 88% ($z = 31.9$). They are wide because the real-time configurations hold few values of S below zero. The Wilson intervals [S3] on the rates themselves are disjoint in every pair.

Every campaign ran both brokers, 25 runs a configuration on each, and the rates above are Kafka's, the half the first analyses read. Redis's factors are in the paper's Table II, and its real-time rate settled at 0.0176–0.0255. Wilson's and Katz's intervals treat messages as independent, while values below zero cluster within a run (S3.4), so the paper's intervals resample whole runs instead. On those, every pair's two rates stay apart on both brokers. On Kafka several pairs' factors have no upper bound, because their real-time values below zero sit in a few of the 25 runs, and a resample can miss them all.

E-A7 also registered that real-time priority would cut the thread's waiting probability at least tenfold. Its scheduler counters show the waiting probability falling 1.7- and 2.2-fold at its two loads, and the mean wait 3- to 5-fold, against the falls in the rate that Table S6 gives for E-A7. The prediction failed. Counters that report means cannot see a tail event, so these campaigns leave the factorization of Equation 3 untested. The kernel trace of S3.5 times the wait directly.

S3.2. One utilization, placed two ways, gives two rates

A concentrated load keeps k of the $C = 8$ cores fully busy and leaves the other $C - k$ free, while a spread load keeps every core busy part of the time and leaves none free. If values of S below zero come from a thread that cannot find a free core, the spread load should produce more of them at equal utilization, and on Kafka it does (Table S7, Kafka's runs).

At $k = 6$ both placements sit at $\rho = 0.7531$, identical to four decimals, on one machine in one hour, and their rates differ 2.07-fold ($z = 10.3$; bootstrap over runs, 95%: 1.36–3.60). No function of ρ can do that, since a function returns one value for one input. The replication E-A6b reproduces the cell at the same ρ and a factor of 2.02, and the direction at $k = 5$. At $k = 7$ the difference is small, since one free core in eight is nearly as useless as none, and only the replication resolves it (1.17×, $z = 3.20$). Multi-server queueing already expects placement to matter at fixed ρ, so the contrast refutes not queueing itself but the single-parameter form, in which ρ alone fixes the rate.

On Redis, in the same runs, the order reverses. At $k = 6$ the spread load's rate is 0.65 times the concentrated load's [bootstrap over runs, 95%: 0.55–0.78], and 0.62 times in the replication [bootstrap, 95%: 0.51–0.76]. One utilization still gives two rates, which is what refutes a rate set by utilization alone, but the explanation above fits Kafka only. We have not tested why Redis's clients fare worse with two cores free. One candidate is its producer's pool of worker threads (Table S3), which may need more cores than a concentrated load leaves.

S3.3. A longer path gives fewer values below zero

The same waits must outlast a longer interval to put S below zero, so a longer path should give fewer such values. Padding the payload lengthens the path without touching the scheduler. In E-A10 the median of S rose 74.3-fold over 4 levels while Kafka's negative-span rate fell 3.8-fold [Katz 95%: 3.3–4.5], monotonically, and a repeat on another day, E-A10b, agrees (Table S8). On Redis the rate fell 7.9-fold [bootstrap over runs, 95%: 6.2–10.4], and 5.7-fold in the repeat, though not at every step there. Serialization pushes utilization slightly up, which would raise the rate, so the fall is measured against its own confound. An account in which stress on the host causes the values below zero predicts the opposite direction.

TABLE S7
AT $k = 6$ ONE UTILIZATION GIVES TWO RATES, IN TWO CAMPAIGNS. CONCENTRATED LOAD LEAVES $C - k$ CORES FREE; SPREAD LOAD DUTY-CYCLES ALL OF THEM AND LEAVES NONE. EVERY CELL IS KAFKA'S 2,497 TO 2,985 MATCHED EVENTS OVER 25 RUNS, AND z IS A TWO-PROPORTION TEST BETWEEN THE TWO PLACEMENTS OF A ROW, WHICH TREATS MESSAGES AS INDEPENDENT. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES. ACHIEVED ρ AT $k = 6$ IS 0.7531 IN ALL FOUR CONFIGURATIONS; AT $k = 5$ IT IS 0.6284/0.6250 AND 0.6284/0.6259, AND AT $k = 7$ 0.8778/0.8809 AND 0.8778/0.8822.

	E-A6 (original)			E-A6b (replication)			
k/C	concentrated	spread	z	concentrated	spread	z	factors
5/8	0.0201	0.0379	4.09	0.0128	0.0584	8.83	1.88×, 4.55×
6/8	0.0824	0.1709	10.27	0.0706	0.1429	8.34	2.07×, 2.02×
7/8	0.2281	0.2415	1.22	0.2359	0.2752	3.20	1.06×, 1.17×

TABLE S8
SLOWER PATH, MORE RELIABLE MEASUREMENT, IN TWO CAMPAIGNS. PAYLOAD PADDING IN TWO CAMPAIGNS. THE MEDIAN OF S SPANS 74.3× AND 77.3× WHILE THE RATE FALLS 3.85× AND 4.04× [3.44–4.74], BOTH MONOTONICALLY, WITH UTILIZATION HELD TO THE ρ SPAN SHOWN. EACH CELL IS 2,408 TO 2,980 EVENTS. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES.

	E-A10		E-A10b		
Padding (B)	Median S (ms)	Rate	Median S (ms)	Rate	ρ span
0	0.697	0.2627	0.670	0.2749	0.881–0.882
4,096	2.378	0.1909	2.199	0.1944	0.882–0.882
65,536	14.469	0.0901	13.973	0.0928	0.885–0.885
262,144	51.768	0.0683	51.795	0.0681	0.893–0.895

S3.4. Load mostly changes how often the thread waits

A thread that reads the clock is either running, and reads it late by a narrow jitter, or waiting for a core. From an idle host to the heaviest load before its processors are fully busy, E-A3's negative-span rate grows 61-fold while the jitter's width grows only 5-fold (Table S9). Load therefore moves mostly the waiting probability p of Equation 3, the chance that the thread is waiting, and little the running state's jitter. A scale family, in which load only widens one distribution, has one parameter and cannot move the two this differently. The tails agree. The two states change p, so the tail curves of two loads should shift vertically and stay parallel on a log scale, where a scale family would rescale them horizontally. Over tail estimates backed by at least 20 events, the log ratio of two conditions' tail masses varies over thresholds by a median factor of 1.34 across conditions, while the scale-family collapse fails by 23×. That test was exploratory; the registered test of p is the priority manipulation of S3.1.

A wait lasts an interval, so the messages inside one are read late together. A Wald–Wolfowitz runs test on each run's sequence of signs of S finds strong clustering at every load, even with no background load. Coarse kernel timestamps would corrupt messages independently and give $z \approx 0$.

Equation 3 describes how the rate moves between conditions, not single messages. Read with the measured D in place of the margin, it would have the rate fall as D grows inside a condition, and it does not. We counted each condition's messages in bins of D, 8 to a decade with at least 172 messages a bin, a count fixed before any bin was read. The rate rises from one bin to a longer one, with disjoint Wilson intervals [S3], in 24 of 56 testable conditions, and in 21 to 29 over four, eight and twelve bins a decade. The rises sit in the runs the sign check rejects (19 of 40 conditions), not in those it passes (2 of 50), whose rates are low. Within a run D and A correlate, with a median coefficient of 0.80 over our groups, so a long delivery comes with a long publish latency, and a message chosen for its long D is likely to have had its acknowledgment read late as well. The margin itself cannot be read message by message. Between conditions, where padding lengthens it at fixed load, the rate falls as the equation says (S3.3).

S3.5. A kernel trace predicts the rate unfitted, and changes it

The negative-span rate is close to the probability that a wait for a core outlasts 0.5 ms, our estimate of the margin in these runs, measured without reference to S. With `bpftrace` on the `sched_wakeup` and `sched_switch` tracepoints we recorded the delay between a thread that reads the clock becoming runnable and reaching a core, for every such transition in a run. The two quantities share no probe, no estimator and no data. One is a count of values below zero in application records, the other a kernel histogram of scheduler transitions. The trace counts every `python3` thread in a configuration, the producers and consumers of both brokers, so we set it against the rate of the same runs with both brokers pooled. Across the three normal-priority configurations of Table S10 the ratio of traced probability to that rate is 0.84, 1.01 and 1.17, with nothing fitted. The ratios lie above and below one, so no bias smaller than their spread can be resolved.

A first attempt at the traced run, whose probe failed to attach, became an untraced twin. It ran two hours apart with the same design. At normal priority it measured Kafka's rate at 0.273 against the traced 0.272 ($z = 0.1$), and the pooled rates

TABLE S9
SHORT OF FULL UTILIZATION THE RATE GROWS 61-FOLD AND THE JITTER ONLY 5-FOLD. E-A3, NINE LOAD LEVELS, 25 RUNS EACH, REGISTERED. THE JITTER WIDTH IS $\sigma_{\text{core}} = \text{IQR}/1.349$. VALUES OF S BELOW ZERO CLUSTER AT EVERY LOAD (RUNS-TEST z). THE THREE $\rho = 1$ ROWS ARE DISTINCT OVERSUBSCRIPTION LEVELS THAT THE UTILIZATION CANNOT SEPARATE.

Utilization ρ	Jitter width σ_{core} (ms)	Negative-span rate	Clustering z
0.003	0.126	0.004	−6.8
0.253	0.138	0.004	−6.8
0.504	0.180	0.006	−6.8
0.628	0.181	0.024	−6.9
0.753	0.238	0.076	−5.7
0.878	0.631	0.224	−4.9
1.000	1.224	0.371	−4.3
1.000	1.293	0.317	−4.7
1.000	1.710	0.261	−4.4

TABLE S10
KERNEL TRACING PREDICTS THE NEGATIVE-SPAN RATE, UNFITTED, IN EVERY NORMAL-PRIORITY CONFIGURATION. Pr[WAIT $>$ 0.5 MS] FROM THE `SCHED_WAKEUP` AND `SCHED_SWITCH` TRACEPOINTS, AGAINST THE NEGATIVE-SPAN RATE FROM APPLICATION TIMESTAMPS IN THE SAME RUNS: BOTH BROKERS POOLED, AS THE TRACE COUNTS BOTH, AND KAFKA'S ALONE, AS FIRST COMPUTED. EACH BROKER CONTRIBUTES 2,257 TO 2,985 EVENTS A CONFIGURATION. EVERY TRACED REAL-TIME CONFIGURATION READS EXACTLY ZERO ON KAFKA, WHERE THE UNTRACED CONTROL READS 15 IN 2,985, SO THAT ZERO IS THE TRACER'S. GENERATED AT BUILD TIME FROM THE CAMPAIGN FILES.

						Both brokers		Kafka alone	
Campaign	Priority	Load	ρ	Traced wakeups	Pr[wait $>$ 0.5 ms]	Rate	Ratio	Rate	Ratio
E-A9	normal	88%	0.8822	551,956	0.181	0.214	0.84	0.272	0.66
E-A9	real-time	88%	0.8819	570,591	0.018	0.011	1.63‡	0.000	—
E-A9b	normal	75%	0.7543	709,819	0.168	0.144	1.17	0.164	1.02
E-A9b	real-time	75%	0.7531	641,308	0.016	0.008	1.95‡	0.000	—
E-A9b	normal	88%	0.8825	687,986	0.207	0.205	1.01	0.259	0.80
E-A9b	real-time	88%	0.8822	649,788	0.016	0.009	1.71‡	0.000	—
Untraced control: E-A9's first attempt, probe never attached									
—	normal	88%	0.8812	—	—	0.214	—	0.273	—
—	real-time	88%	0.8809	—	—	0.013	—	0.005	—

‡Kafka's half of the pooled rate reads zero under the tracer: shown, not used.

agree as closely, so there the tracer leaves the rate alone. Each prediction is held against the traced run's own rate all the same, so that a common perturbation would move both quantities together. A rule fixed in advance, the tracer check, compares a traced configuration only if its rate lies within 25% of its untraced comparator. The largest drift is 4% on the pooled rate and 5% on Kafka's, so all three are compared. Against Kafka's rate alone, as first computed, the ratios are 0.66, 0.80, 1.02. They are lower because the trace counts the threads of both brokers and Kafka's rate is the higher.

At real-time priority the tracer does change the rate, and only Kafka's. Kafka's traced real-time configurations read exactly zero where its untraced control reads 15 in 2,985, while Redis's read 1.6–2.2%, close to its untraced real-time rate. The pooled real-time rate is therefore Redis's halved, and the traced probability stands to it at 1.6–2.0. There the tracer silenced half of what the trace is compared with, so these ratios are shown and not used.

S3.6. Rival explanations, each excluded or bounded

Five rivals fall to the design and the manipulations. One clock excludes clock skew between hosts, the cause a concurrent study assigns to such values (postmortem, S33). One utilization at rates twofold apart excludes utilization (S3.2). Real-time priority excludes timer noise, which would delay a thread whatever its scheduling class (S3.1), and a longer path that lowers the rate excludes stress on the host (S3.3). Coarse kernel timestamps fall to the runs test of S3.4. Five more need an argument each, and the last of them is bounded, not excluded.

The second thread: Kafka's client reads the clock for the acknowledgment on its I/O thread when the delivery callback fires, and Redis's on the calling thread the moment `XADD` returns (Figure 1). The hand-off is not the mechanism. Redis, which has no second thread, has values of S below zero on 8.71% of its messages against Kafka's 8.87%, and on more of its runs (90% against 65%). Its calling thread, returning from a blocking socket read, must also be woken and given a core before it reads the clock.

The hypervisor: Both testbeds are virtual machines, so steal time, the time a virtual processor is ready to run but waits while the hypervisor runs another machine, is bounded, not excluded. Two observations disagree with steal time as the cause. Real-time priority is a setting inside the virtual machine, which steal time would not yield to, yet it cuts the rate 7- to 80-fold on Kafka and 6.0- to 7.8-fold on Redis (S3.1). And the upper mode of the traced waits shares its bucket with the base slice

that the virtual machine's own kernel derives (S3.7), not with any quantity the hypervisor exposes. The Oracle runs did not record steal time. The registered campaign recorded it from `/proc/stat` in 8,092 runs and found none, but we did not establish whether the Azure hypervisor reports steal time to its machines, so that reading does not exclude it either.

The interpreter lock: The threads that read the clock are CPython, which releases the lock around blocking I/O and retakes it on return, so a timestamp waits for a core and then for the lock. The kernel trace cannot see the second wait, and neither priority nor the size of the waits tells the two apart. The direction of the trace's error settles it. A second wait in series with the run queue would make the trace underestimate the rate, but its ratios to the rate of the runs it traced, 0.84, 1.01 and 1.17, lie on both sides of one. The registered campaign also found values of S below zero in Kafka's own Java client, which has no interpreter lock, at a lower rate (S6.2).

The clock: Clocks that disagree across virtual CPUs [S4] are excluded twice. First, the platform probe's smallest observed increment of `clock_gettime(CLOCK_MONOTONIC)` was 90 ns, an upper bound on the cost of one read. That excludes `acpi_pm` by arithmetic, since its tick is 279 ns, and `hpet` by its published read cost near $1.2\,\mu$s. What remains, `kvm-clock` or `tsc`, carries the coherence guarantee. A `kvm-clock` that fast is on the vDSO path, which `kvmclock.c` enables only when the hypervisor sets `PVCLOCK_TSC_STABLE_BIT`, the flag that guarantees monotonic time across CPUs. A KVM guest selects `tsc` only when the kernel finds it constant, non-stop and stable, and abandons it if it later falls out of step between CPUs. Which of the two ran is unrecorded, and the published read costs vary about twofold, against an order-of-magnitude margin. Second, an offset between clocks large enough to produce the deepest value of S, -99.8 ms, would have driven D far below zero. The smallest D observed, $220\,\mu$s, bounds any such offset 455-fold below what that value needs.

The broker's reply order: Both readings follow one write, but the broker then answers twice, with the acknowledgment to the producer and the message to the consumer, and nothing here establishes the order of the two. A broker that delivered the message first would put S below zero with no wait at all. Kafka runs with `acks=all` (Table S3), so both of its replies wait for the same commit, and Redis produces both in the same pass of its event loop. Two observations bound this rival without excluding it. In the registered campaign, 79.4–92.0% of the readings below zero were mostly spent waiting for a core (S6.2), so the reply order could account at most for the rest. And real-time priority, which cannot reorder the broker's replies, cut the rate in every pair (S3.1), so the residual rate at real-time priority is where it could hide. A capture of both connections' packets on the client host would settle it, and we did not make one.

S3.7. The waits' upper mode survives coarsening, and its bucket holds the derived slice

The traced waits have 3 local maxima (Figure S1), and the upper one survives a change of bucket edges. How many maxima a histogram shows depends on its bin edges as well as on its counts, and these edges came from `bpftrace`. The per-event waits were not retained, so coarsening is the only test available. Merging adjacent buckets doubles the bin width, and starting the merge at the first bucket or the second puts the edges in different places. The spectrum has 3 maxima on the tool's own 16 buckets, 3 again under both phasings at 2 times the width, and fewer at 4 times, so the count holds to within a factor of 2 in bin width.

Buckets that double in width favor the upper ones, so part of the upper mode is the buckets' doing. Waits spread evenly up to the slice and then up to a tick, as Equation S3 has them, would make the bucket that holds the slice 1.2 times the bucket below it. The traced bucket is 4.5 times it, so the waits gather at the slice more than an even spread would put them there. We have not explained the middle mode, at 128–256 μs.

Above the upper mode the counts collapse. They fall 5.0-, 3.4- and 4.7-fold over the next three octaves and then 357-fold, each octave a doubling of the wait, whereas a power law would fall by the same factor in every octave. Power laws fitted below the mode and above it both fail a bootstrap goodness-of-fit ($p < 0.0004$ and $p < 0.0004$, over 2,500 replicates each).

The slice is derived from the kernel's rule and the campaigns' own data. Each placement condition loads k cores and records the achieved utilization, so k/ρ recovers 8 cores six times over, with a spread of 0.05. Equation 4 then gives $4 \times 0.70\,\text{ms} = 2.8$ ms. Upstream Linux 6.8 used $s_0 = 0.75$ ms; the change to 0.70 ms [S5], first released in 6.15, reached this kernel through Ubuntu's stable updates (linux-oracle-6.8 6.8.0-1043, January 2026, with no revert before the 6.8.0-1057 every cloud run recorded), and the registered campaign's Azure kernels, from the same updates, read back the same constant. Either value puts the slice in the 2–4 ms bucket. The build sets `CONFIG_HZ` $= 1000$, a 1 ms tick. With run-to-parity, a woken thread that loses the wakeup-preemption check waits for the running thread to finish its slice, and since a task notices its deadline only at the next tick, waits end in a band a tick wide. No scheduler tunable is written anywhere in the archived campaigns, so the derived value is the one that applied. That is strong evidence, short of proof, because the slice was not read off the hosts. The registered campaign later timed the wait itself and found it to be the rest of a slice (S6.3).

S3.8. Timing from the publish call keeps the receiving thread's wait, under a millisecond on the mean

A latency timed from the publish call carries the receiving thread's wait for a core. By Equation 1, $D = S + A$ for every message, so the acknowledging thread's wait cancels: it lengthens A by as much as it shortens S. The receiving thread's wait does not cancel. The consumer reads its clock only after its thread has been woken and given a core, so every D holds that

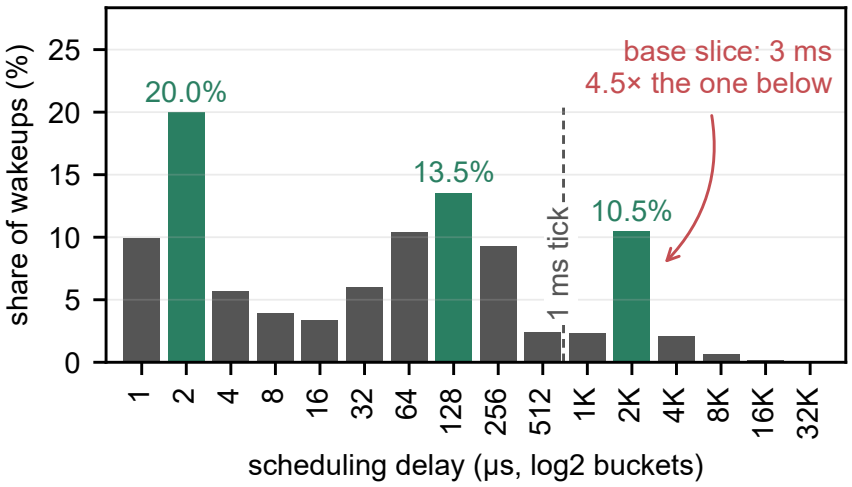


Fig. S1. **Traced scheduling delays are trimodal.** 551,956 wakeups of E-A9's normal-priority configuration at 88% load (Table S10), `bpftrace` $\log_2$ buckets, modes shaded. The jitter peaks in the 2–4 μs bucket, which alone holds 20.0% of all wakeups. A second mode at 128–256 μs holds 13.5%. The third, at 10.5%, sits in the bucket holding the scheduler's derived base slice of 2.8 ms. The count of modes survives a 2× widening of the buckets in both phasings.

TABLE S11

THREE MEASUREMENTS PUT THE RECEIVING THREAD'S WAIT AT 0.41–0.94 MS OF THE MEAN LATENCY. ONE ROW FOR EACH BROKER AND LOAD. METHOD 1 IS THE FALL IN MEAN D WHEN THE CONSUMER ALONE RUNS AT REAL-TIME PRIORITY. METHODS 2 AND 3 ARE THE FALLS, FROM NORMAL TO REAL-TIME PRIORITY, IN THE MEAN TRACED WAIT AND IN THE MEAN DELAY FROM THE KERNEL'S RECEIVE TIMESTAMP TO THE TIMESTAMP. EACH FIGURE IS A SETUP'S MEDIAN OVER ITS RUNS. THE CLIENT'S WORK IS THE MEDIAN, OVER TRACED RUNS, OF THE DELAY BEYOND THE TRACED WAIT.

	Added to the mean of D (ms)			Largest over	D, normal → real-time priority (ms)		Client's
Broker, load	Method 1	Method 2	Method 3	smallest	Median	Ninetieth percentile	work (ms)
Kafka, 75%	0.43	0.70	0.64	1.64	2.03 → 1.82	5.21 → 4.27	0.37
Kafka, 88%	0.94	0.91	0.92	1.04	2.52 → 1.92	5.66 → 4.53	0.39
Redis, 75%	0.47	0.50	0.47	1.07	0.89 → 0.84	3.06 → 0.93	0.11
Redis, 88%	0.43	0.62	0.41	1.49	0.87 → 0.83	2.86 → 0.89	0.12

wait, which is never negative and which the sign check of Section VI cannot see. After the registered campaign we measured it three ways in the same runs, by a plan and four predictions written before any run.

The runs used the registered campaign's first x86 pair (S6.1), both brokers, 75 and 88% load and the base slice at 3 ms, with no delay added to the network. Eight setups, each broker at each load with the consumer at normal or at real-time priority, ran in six shuffled rounds. Three runs added before anything was read gave every setup at least three traced runs, and all 51 runs passed the integrity rule. Method 1 runs the consumer alone in the real-time class at priority 80 and measures how much that lowers D at unchanged load. Method 2 traces the scheduler in a random half of the runs and measures the receiving thread's own wait for a core before each timestamp, as S3.5 did for the acknowledging thread. Method 3 captures the receiver's packets in every run with the kernel's nanosecond timestamps, and times each message from the packet that carries it to the consumer's timestamp. That delay includes the wake-up, the wait and the client's own work.

The three estimates run from 0.41 to 0.94 ms (Table S11) and lie within a factor of 1.5 of each other at three of the four points. The wait lives in the tail. At normal priority the median message waited 18–46 μs and the mean 0.51–0.92 ms, while the ninetieth percentile reached 2.13–2.92 ms. Real-time priority brought the mean wait down to 10–11 μs. On Redis it cut the ninetieth percentile of D to under 1 ms and left the median almost where it was.

The first prediction held. On both brokers at both loads the wait's mean at normal priority lay between 0.3 and 1.2 ms and its ninetieth percentile above 1.5 ms, and real-time priority took the mean below 0.1 ms. The other three failed, each on Kafka. There real-time priority lowered the ninetieth percentile of D by less than the 1 ms predicted at 75% load, and moved the median by more than the 0.3 ms allowed at 88%. Method 3's delay was at least as long as the traced wait for 99.7% or more of every traced run's messages, against the 95% predicted. But Kafka's client then worked longer than the 0.3 ms predicted before its timestamp, where Redis's stayed under it. At 75% load Kafka's first two estimates lie further apart than the factor of 1.5 allows. We have not tested why Kafka differs.

Three things qualify the figures. The tracer is not a free observer: at normal priority the traced runs' mean D ran 5–22% above the untraced runs', so the second method may read the wait somewhat high. In five runs a message arrived more than 150 ms late, three of them during stalls of the whole driver, of the kind the registered campaign marks as paused (S6.2). A setup's value is a median over its runs, which sets them aside. And three of the four setups at real-time priority hold only two

untraced runs, too few to judge the tracer there. The plan, the register of every run and the reading are in the repository under `docs/results/recv_wait`, read by `scripts/recv_wait.py`.

S4. The positivity filter and the retention law in full

The benchmark's printed latency describes only the samples its publish schedule lets through the positivity filter, and each link in that claim can be checked against a source line, a counter or a registered prediction. Every benchmark run below is in embedded mode, with the whole benchmark in one JVM.

S4.1. Two source lines make the filter, and counting what they drop changes no printed number

What the positivity filter drops can be counted without changing the filter. In `LocalWorker.java`:294 the benchmark computes its end-to-end latency as

```
MILLISECONDS.toMicros(now - publishTimestamp)
```

from two millisecond timestamps, so the value, though in microseconds, is a whole number of milliseconds. In `WorkerStats.java`:95 the positivity filter,

```
if (endToEndLatencyMicros > 0)
```

decides whether that value reaches the histogram. The message has already been counted as received, but nothing counts what the condition drops.

We add counters in an `else` branch of that condition and change nothing else: not the latency computation, not the filter, not any reported statistic. The benchmark's output is therefore exactly what it would have been unpatched, and its printed summary can be set beside the retention behind it (Fig. 4b of the paper). The four counters, for zero, negative, most negative and kept samples, separate sign. A value below zero and an end-to-end latency shorter than a millisecond both fail a `> 0` test, and one unsigned total cannot tell them apart (S4.4). The patch is `external/omb/workerstats_discard_counter.patch` in the artifact.

The counts are exact. The counters are shared across the JVM and their totals printed once at shutdown. A run that fails discards nothing, and its zero count would look measured, so a validity gate writes it as invalid, with its reason and no count. Unlike the benchmark's histograms, our counters include the one-minute warmup, yet a matched sweep with the warmup disabled leaves retention unchanged. The campaign totals 223 instrumented runs, swept one axis at a time with background load, message size, producer rate and warmup otherwise fixed.

S4.2. The phase of the publish schedule sets retention, as a 1970 counter note already showed

Where each publish instant falls within a step of the clock decides whether its sample survives. An end-to-end latency shorter than the step τ survives only if it crosses a grid instant, where the timestamp changes value. Otherwise both timestamps round down, or floor, to the same value, and the positivity filter deletes the zero difference (Fig. 4a of the paper). When phases are uniform, retention is $\theta = T_{\mathrm{true}}/\tau$, which is Equation 5 of the paper.

A paced producer does not draw its phases uniformly. If its publish interval T_{pub} satisfies $T_{\mathrm{pub}}/\tau = a/q$ in lowest terms, it visits only q distinct phases, so its retention is a whole count out of q. Call the retentions $0, 1/q, \ldots, 1$ the vertices of the configuration's q-lattice, and the interval between neighboring vertices, $100/q$ percentage points wide, a lattice step. At an incommensurate rate, whose ratio has no small denominator, the vertices are dense and retention is θ. Elsewhere, replicates of one configuration land on the vertices that bracket θ, and their spread obeys

$$\text{spread} \longrightarrow \begin{cases} 100/q & \text{when } \theta \text{ sits mid-step,} \\ 0 & \text{when } \theta \text{ sits on a vertex.} \end{cases} \tag{S1}$$

The step width is thus an upper bound on the spread, reached only when replicates realize both bracketing vertices. Retention at the incommensurate rates puts T_{true} at 0.49 ms, half a millisecond to within replicate noise (Section V-C). One half is a vertex of every q-lattice with even q and of none with odd q, so the spread rule predicts full step width for odd denominators and almost none for even ones.

Hewlett-Packard's Application Note 162-1 [S6] carries this arithmetic for a counter timing an interval. For an interval of $Q + F$ clock periods it gives $\Pr[R = Q{+}1] = F$, which is Equation 5, and the denominator of the ratio of repetition rate to clock rate in lowest terms, our q, fixes how many distinct readings the counter can produce, so averaging N readings improves resolution by at most a factor $1/q$, not $1/\sqrt{N}$. Its cure, phase jitter on the repetition rate, becomes for a benchmark randomized publish instants (the paper's Table IV).

TABLE S12
REPLICATE SPREAD FOLLOWS THE PHASE DENOMINATOR q IN 10 OF THE 12 COMMENSURATE CONFIGURATIONS. PAYLOAD, LOAD, DURATION AND HOST ARE FIXED. THE POSITION IS THE DISTANCE FROM THE MEASURED CONTINUOUS VALUE TO THE NEAREST VERTEX, AGAINST HALF THE STEP WIDTH. A CONFIGURATION IS PREDICTED FULL WHERE ITS POSITION EXCEEDS A HALF, AND SHOWS FULL WHERE ITS SPREAD EXCEEDS HALF THE STEP WIDTH. THE DAGGERED ROWS ARE THE TWO MISSES, RESOLVED IN TABLE S13. INCOMMENSURATE CONFIGURATIONS HAVE NO LATTICE STEP. THEIR SPREADS OF ABOUT THREE POINTS ARE THE NOISE FLOOR.

rate	T_{pub}/τ	q	n	width	pos.	predicts	spread	shows
1000/s	1/1	1	5	100.0	0.98	full	99.4	full
500/s	2/1	1	5	100.0	0.98	full	99.5	full
250/s	4/1	1	5	100.0	0.98	full	98.7	full
400/s	5/2	2	5	50.0	0.04	flat	17.6	flat
600/s	5/3	3	6	33.3	0.95	full	32.7	full
300/s	10/3	3	10	33.3	0.95	full	31.2	full
800/s	5/4	4	5	25.0	0.07	flat	7.2	flat
1250/s†	4/5	5	5	20.0	0.91	full	7.6	**flat**
625/s	8/5	5	10	20.0	0.91	full	19.3	full
875/s	8/7	7	5	14.3	0.87	full	10.7	full
700/s†	10/7	7	5	14.3	0.87	full	6.5	**flat**
900/s	10/9	9	5	11.1	0.84	full	6.9	full
incommensurate — phase set effectively continuous								
333/s	1000/333	>64	5	—	—	≈ 0	4.5	—
383/s	1000/383	>64	5	—	—	≈ 0	2.8	—
457/s	1000/457	>64	5	—	—	≈ 0	2.1	—
611/s	1000/611	>64	5	—	—	≈ 0	3.1	—
889/s	1000/889	>64	5	—	—	≈ 0	1.7	—
1053/s	1000/1053	>64	4	—	—	≈ 0	3.0	—
1219/s	1000/1219	>64	4	—	—	≈ 0	4.5	—

S4.3. Retention keeps to its lattice, and a registered change of message size moved one configuration onto a vertex and off it

The phase account predicts each configuration's behavior from its denominator alone. Its starkest case is $q = 1$: where the publish interval is a whole number of milliseconds, every publish has the same phase, and replicates of one configuration spread over almost the whole range, while those at incommensurate rates agree to within a few points (Table S12). The spread rule of Equation S1 holds in 10 of the 12 commensurate configurations. A spread above half the step width counts as full, so the test needs no tolerance. The two misses, 700 and 1250 msg/s, are predicted full and measured flat. Spread is a range over replicates that can each land on only two vertices, so a configuration shows its full width only if its replicates land on both. At $q = 5$ and $q = 7$ the upper vertex is the less likely of the two, so five replicates can all fall on the lower one even when the lattice holds exactly, and spread cannot tell that outcome from a refutation.

For that reason we built a second test after the campaign had run. It was not registered, and its first results were committed in the same change as its code. For each configuration, $\bar{d}$ is the mean distance of its replicates to the nearest vertex of its q-lattice, divided by the step width. The null is a continuum, in which replicates are drawn from a Normal distribution centered on the retention the incommensurate rates imply at the configuration's rate, with a measured standard deviation. Neither parameter is fitted to the configuration under test. The one-sided p-value comes from 4,000 Monte Carlo draws at the observed number of replicates, and is Holm-corrected across all 12 configurations. Of these, ten are powered, meaning the null and the lattice predict different distances, and nine of those reject the continuum at $\alpha = 0.05$ after correction (Table S13), among them both configurations the spread rule missed, 700 msg/s at 0.009 and 1250 msg/s at 0.003. The tenth, 900 msg/s, rejects before correction ($p = 0.044$) and not after ($p = 0.131$), so we report it as unresolved. The two unpowered configurations, 400 and 800 msg/s, have lattices that pass through θ, where the two hypotheses coincide.

The registered test that manipulates the physics is the payload flip. Payload moves T_{true}, and so moves θ within its lattice step, with nothing else changed. Measured at an incommensurate rate (Table S14), 32 KB puts θ almost on the $2/3$ vertex of the $q = 3$ lattice and 64 KB puts it mid-step. The prediction was that this one manipulation would move a 300 msg/s configuration onto a vertex and off it again.

The spread flipped from full to flat and back, as predicted (Fig. S2). At 32 KB the five replicates spanned 62.06–75.69%, a spread of 13.6 points, and at 64 KB they spanned 73.06–99.78%, a spread of 26.7 points. Both cross the flat–full boundary at half the step width, 16.7 points, in the predicted direction. One registered detail failed: the 32 KB configuration pins at 69.2%, not on the vertex at 66.67%. The flip was one of six predictions registered with falsifiers before its campaign. It was confirmed, three were refuted by their falsifiers, one could not be evaluated, and one split (postmortem, S1.7).

S4.4. Two floored clocks disagree by exactly one millisecond, so the sign of a discard names which kind of timestamp failed

The sign of a discarded sample says which clock failed, and the benchmark's two drivers show both kinds. The Kafka driver's 214 runs discarded 10,913,263 samples, and not one was negative. The Redis-driver replication's 9 runs carry all 41,403 of the campaign's negatives, every one exactly $-1000\,\mu$s.

TABLE S13
RETENTION SITS ON THE q-LATTICE IN NINE OF THE TEN POWERED CONFIGURATIONS, TESTED AGAINST A CONTINUUM NULL. HERE $\bar{d}$ IS THE MEAN NORMALIZED DISTANCE TO THE NEAREST VERTEX, AND THE NULL COLUMN IS ITS MEAN UNDER THE SIMULATED NULL. THE COLUMN p_{Holm} IS A MONTE CARLO p-VALUE (4,000 DRAWS), HOLM-CORRECTED WITHIN ALL 12 CONFIGURATIONS; IT DECIDES THE VERDICT. FLAT (COINCIDENT) MARKS CONFIGURATIONS WHERE THE TWO HYPOTHESES PREDICT ALIKE AND THE TEST HAS NO POWER.

rate	a/q	n	$\bar{d}$	null	p_{Holm}	verdict
1250/s	$4/5$	5	0.131	0.351	0.003	grid
1000/s	$1/1$	5	0.003	0.479	0.003	grid
900/s	$10/9$	5	0.265	0.340	0.131	not resolved
875/s	$8/7$	5	0.209	0.382	0.003	grid
800/s	$5/4$	5	0.104	0.058	1.000	flat (coincident)
700/s	$10/7$	5	0.329	0.416	0.009	grid
625/s	$8/5$	10	0.112	0.447	0.003	grid
600/s	$5/3$	6	0.034	0.469	0.003	grid
500/s	$2/1$	5	0.043	0.490	0.003	grid
400/s	$5/2$	5	0.093	0.020	1.000	flat (coincident)
300/s	$10/3$	10	0.221	0.468	0.003	grid
250/s	$4/1$	5	0.181	0.489	0.003	grid

TABLE S14
AT A RATE INCOMMENSURATE WITH THE MILLISECOND, RETENTION READS OFF THE END-TO-END LATENCY. RETENTION AGAINST PAYLOAD AT 457 MSG/S, WHERE SAMPLES STAY DEPHASED. THE IMPLIED T_{true} INVERTS EQUATION 5. UP TO 8 KB THE ADDED DELAY IS COMPARABLE TO THE REPLICATE NOISE; THE 32 AND 64 KB LEVELS CARRY THE RESULT.

Payload	Retention (3 reps)	Median	Implied T_{true}
200 B	51.08, 52.03, 52.88	52.03%	0.520 ms
1 KB	50.11, 50.32, 51.47	50.32%	0.503 ms
2 KB	52.13, 54.62, 54.77	54.62%	0.546 ms
4 KB	51.86, 54.17, 56.24	54.17%	0.542 ms
8 KB	54.92, 55.22, 57.89	55.22%	0.552 ms
32 KB	68.01, 68.44, 69.40	68.44%	0.684 ms
64 KB	84.21, 85.36, 86.82	85.36%	0.854 ms

The drivers explain the difference. The Redis driver takes its publish timestamp from `entry.getID().getTime()` in `RedisBenchmarkConsumer.java`:72, the stream-entry identifier the Redis server assigns on `XADD`. The Kafka driver uses a producer-set timestamp. So the Redis span subtracts a server-side timestamp from a consumer-side clock reading, across two hosts, with both floored to the millisecond, while the Kafka span floors two timestamps whose difference is taken inside one JVM.

Two clocks floored to the same resolution and offset by less than one millisecond can disagree by one millisecond and by no other amount. The only negative a floored cross-host difference can produce is therefore $-1000\,\mu$s, and every one of the 41,403 negatives is that value. Clock synchronization was clean at every logged minute, so this failure needs no misbehaving clock, only two clocks whose readings are floored. A difference taken inside one process has no second clock to disagree with, which is why the Kafka discards hold no negative. The same positivity filter thus hides a loss of resolution under Kafka and a clock failure under Redis, and an unsigned discard counter could not tell them apart.

S4.5. An independent harness reproduces the deletion, and the benchmark's distributed mode, which failed on the cloud testbed, ran in the registered campaign

The deletion does not depend on the benchmark's code. We reproduced the positivity filter in a Python harness that shares no code with the benchmark, with the same timestamp, subtraction and filter. On one clock it recorded 0 negative differences across 905,040 samples in 12 runs, as a causal chain requires, and the grid reappeared intact. Across two hosts with disciplined clocks, 12 runs again recorded 0 negatives in 905,040 samples and an intact grid, but the deleted fraction followed the drift of the clock offset. Of the three rates, only the incommensurate one, 457 msg/s, has no lattice spread wide enough to hide the drift. Over four replicates there, cross-host retention wandered from 13.4 to 26.9%, while on one clock the same configuration's replicates spanned 0.98 points.

On the cloud testbed the benchmark's own distributed mode, where its subtraction spans two clocks, could not be measured. It never completed a run in eleven attempts: eight at three background-load levels, including none, and three more run with full diagnostics. The unloaded attempts failed like the loaded ones, which refutes our own hypothesis that processor starvation was the cause, and the three diagnosed attempts failed identically, in the coordinator-to-worker path before any workload started. The first attempt, made before the validity gate of S4.1 existed, wrote a vacuous zero that the artifact keeps marked invalid, and the gate refused every later attempt with its reason. On that testbed the distributed mode's exposure to the deletion therefore

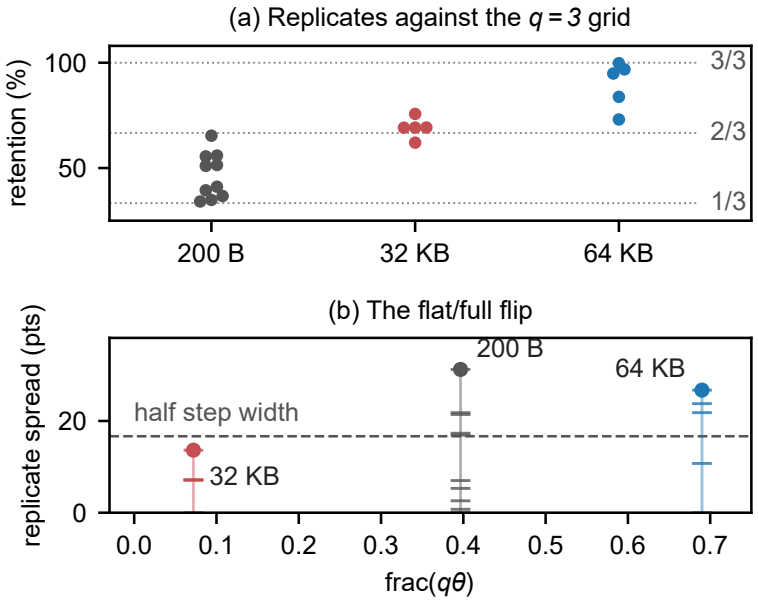


Fig. S2. **One manipulated variable moves a configuration onto its lattice and off it.** (a) Replicates at 300 msg/s ($q = 3$) by payload, against the thirds lattice: 200 B straddles both branches, 32 KB pins near the $2/3$ vertex, and 64 KB re-opens to the upper step. (b) Replicate spread against $\mathrm{frac}(q\theta)$, each stick showing the replicates its spread is made of, with θ each configuration's own mean retention in this campaign. The registered prediction placed 32 and 64 KB at 0.055 and 0.56 from the payload sweep's θ. The flat–full boundary at half the step width is crossed in both directions.

rested on reading its source. The timestamp is written on the producer's host and read on the consumer's, and the positivity filter drops every difference at or below zero.

On the registered campaign's machines the distributed mode ran unmodified in October 2026, in the campaign's tools block (S5). A coordinator and two workers, one producing and one consuming, ran on one machine against a Kafka broker on the other. The benchmark's median followed a delay added to the path. We moved the consuming worker's clock back at two offsets of a few milliseconds, and the filter dropped whatever then computed to zero or below. In three of the four runs at the smaller offset the median fell to 1 ms, the smallest value a surviving sample can take. Elsewhere only the slow tail survived, and the median rose to the tail's; in one run nothing survived to print. The unmodified benchmark counts nothing it drops, so these runs show its filter at work across two processes but do not measure the deletion, which rests on the embedded mode and on the harness above.

S5. Tools Read at Source, Tools Run, and Public Reports

Tools read at source. Eight of the eleven tools we read at source time messages, and seven of those cut a value at or below zero out of a figure they report without counting it. One of the seven, `emqtt-bench`, keeps the value in its histogram and leaves it out of the average it prints. The remaining one, Kafka's bundled pair, never measures it (the note on `EndToEndLatency` below). Counted by whether the sample itself is lost, eight of all eleven dispose of it, `fio` and `btt` among them. Every verdict traces to a line of source a reader can check. Each row of the registry file, `docs/results/external/harness_registry.csv`, carries one source line, the file and symbol it came from, the upstream URL and the date it was read. The class labels are not stored. They are recomputed on every build and every test run by the classifier that read the OpenMessaging Benchmark a month before the other tools were audited (`classify()` in `scripts/audit_external_harness.py`), so a reader who disagrees with a verdict can check the line against the pattern that matched it. Its patterns were written against JVM sources and had to be extended before they could read an Erlang guard or clock call, or librdkafka's guard in C, and they see only what they recognize, so the registry's count of disposals is a lower bound for the field.

We call a tool's own conditional on the sign of an interval its *guard*. A guard is a deletion only if what it guards is the sample itself, not a counter. Table S15 is the registry, one row per span because one tool can treat two spans differently, generated from the registry file at build time. Its sources, in row order, are [S7], [S7]–[S10], [S10]–[S16], [S16], [S17].

The eight silent disposals take three forms. A filter drops what its comparison excludes and reports no count of what it removed. The test > 0 deletes every sample that computes to exactly zero, while ≥ 0 keeps those and drops only values below zero. The benchmark and librdkafka's `rdkafka_performance` filter at > 0. Apache Pulsar's performance client guards at ≥ 0, so the zeros the benchmark deletes pile up visibly at zero. A substitution keeps the sample count, so any retention computed downstream reads 100%, which makes it harder to detect than a filter. A library refusal leaves the sample unrecorded because the tool never checks whether its library rejected it. Truncation is a separate loss: it keeps every sample but loses

TABLE S15
ELEVEN TOOLS, READ AT SOURCE. THE SPAN EACH MEASURES, THE CLOCK IT READS, WHAT HAPPENS TO A VALUE AT OR BELOW ZERO, AND WHETHER THE TOOL COUNTS WHAT IT DISCARDS; ONE ROW PER SPAN WHERE A TOOL MEASURES TWO, AND A DASH WHERE NOTHING IS DISCARDED. THE SOURCE LINE BEHIND EVERY ROW IS IN THE ARTIFACT'S REGISTRY.

Tool	Language, vendor	Span measured	Clock	What happens to the value	Counts
Apache Kafka (bundled)	Java, Apache Software Foundation	end-to-end	nanosecond	truncate to the resolution	no
Apache Kafka (bundled)	Java, Apache Software Foundation	publish	millisecond	truncate to the resolution	no
Apache Pulsar perf	Java, Apache Software Foundation	end-to-end	millisecond	filter, ≥ 0	no
NATS CLI	Go, Synadia	end-to-end	nanosecond	refused by own library	no
OpenMessaging Bench.	Java, Linux Foundation	end-to-end	millisecond	filter, > 0, truncate to the resolution	no
OpenMessaging Bench.	Java, Linux Foundation	publish	nanosecond	—	—
RabbitMQ PerfTest	Java, Broadcom/VMware	end-to-end	nanosecond	substitute a value	no
Rezolus	Rust, IOP Systems	scheduler wakeup	nanosecond	discard	yes
`blktrace/btt`	C, Alan D. Brunelle	block stage	nanosecond	substitute a value	no
`emqtt-bench`	Erlang, EMQ Technologies	end-to-end	millisecond	kept; zero in its printed average	no
`fio`	C, Jens Axboe	I/O completion	nanosecond	substitute a value	no
`rdkafka_performance`	C, Confluent	end-to-end	microsecond	filter, > 0	no
`rdkafka_performance`	C, Confluent	publish	microsecond	—	—
wrk2	C, G. Tene	end-to-end	microsecond	refused by own library	no

precision. The benchmark truncates before it filters, and Kafka's bundled tools only truncate. The notes below carry what the table cannot.

- The OpenMessaging Benchmark appears twice because it treats its two spans differently. It times its end-to-end latency on a millisecond clock and filters it, and times its publish latency in nanoseconds and records every sample (Section V-C).
- librdkafka's `rdkafka_performance` also appears twice, and its two spans differ in the same place. Its consumer subtracts the timestamp the producer wrote into the message from its own wall clock, in microseconds, and keeps the result only if it is above zero and under five minutes. For each value it drops it prints a line saying the value was ignored, but it keeps no count. Its producer times each message from that timestamp to the delivery report on its own clock and records every value.
- The `EndToEndLatency` tool bundled with Kafka cannot produce a value below zero, because it uses one thread, one monotonic clock and a synchronous publish call. It loses the measurement anyway. Each sample prints in floating point, but the `long[]` its percentiles come from is filled by the integer division `elapsed / 1000 / 1000`, so on a path faster than a millisecond its p50, p99 and p999 all read zero. Its synchronous publish call has a second effect: it polls for the message only once the acknowledgment has returned, so a message that reached the consumer first is timed no earlier than its acknowledgment. The receipt-minus-acknowledgment difference this paper measures is held at zero by the order of two calls, and a poll that finds nothing stops the run rather than dropping the message. The companion `ProducerPerformance` takes its publish-to-acknowledgment difference on `System.currentTimeMillis()` and casts it to `int`.
- In `fio` the guard appears in three timing functions, each returning zero for a negative interval under the comment "time warp bug on some kernels?". Nothing records that it fired.
- In `btt`, `tdelta()` returns one nanosecond whenever the stage timestamps are out of order, with no comment, no counter and no mention in the manual page.
- The `wrk2` load generator detects the negative interval, prints a block that begins "We are about to crash and die", and records the value anyway into its own copy of `hdr_histogram.c`. That copy's `assert()`s are live in the shipped build, and it returns `false` for an out-of-range index, a refusal the tool never checks. Either the assertion aborts the run or the refusal drops the sample, and nothing counts the loss.
- In `emqtt-bench` the `> 0` guard protects a counter and leaves the histogram alone, so nothing is deleted from the histogram; a guard that once deleted from it was removed in January 2025. Its printed average keeps a value at or below zero in its denominator and leaves it out of its numerator, as if it were zero.
- In RabbitMQ PerfTest the method that computes the difference, `getDifference`, returns its absolute value, so a negative span enters its metrics as a positive one of the same size, uncounted. The metrics then record only values at or above zero. From version 2.0.0 of 2017 until version 2.25.0 of June 2026 they recorded only values above zero, so in the tool's millisecond mode every trip shorter than a millisecond was deleted, uncounted.
- The NATS CLI appends every difference, negatives included, to the list its minimum, median and maximum are read from, but records its percentile table into `hdrhistogram-go`, whose bound check refuses the index a negative value maps to. The CLI ignores the error, so its percentile table disagrees with its minimum, median and maximum, and nothing counts the difference.
- Rezolus counts the samples its scheduler sampler discards, those whose two timestamps arrived out of order across CPUs, in a metric that names the cause. The counter was added on 12 August 2026, and its other samplers have none.
- Two further tools we read are not in the count, because they do not make the measurement. SPECjms2007 defines Delivery

Time and never reports it, and Redpanda's `rpk` ships no latency benchmark.

Tools run. Reading a tool tells what it does with a value below zero; running it tells whether it ever meets one. The registered law campaign ran 16 benchmarking tools on the first x86 pair, four rounds each, under predictions fixed from their source before any run (its tools block, frozen with the plan; every amendment that touched it is listed in `freezes/README.md`). Eleven ran in September. The plan had registered five more, but its version 15 left them unrun. They ran in October as a second block, under version 33 (`freezes/30-experiment-plan`), which was public before any of their runs. Table S16 lists all of them. Under a delay added in steps, every tool's figure followed the delay at a slope inside the band the plan fixed in advance.

A second test, T2, moved back the clock of the process that subtracts, at two offsets longer than the interval the tool times. A program registered with the plan, the judge, then chose for each run among five behaviors: keeps every value, drops the negatives, drops zero and below, replaces the negatives with zero, and times against one clock. The second block exposed two faults in the judge. First, a tool that kept fewer values than it was asked to publish was ruled out of the behaviors that keep every value, but not out of timing against one clock. Yet a tool on one clock keeps every value too, so a run of Pulsar's client that kept 7 of 3,000 latencies was judged to time against one clock. Second, the judge predicts each behavior's figures from the trips of a reference, our own client for the same server, not from the tool's. A tool with a slow tail of its own can therefore be ruled out of the behavior it shows. Version 34 (`freezes/31-experiment-plan`), public before either block was judged again, corrects the first fault. It also decides how many clocks a tool reads from the offset alone, without the reference. A tool reads two if it lost values, or if its average, median or minimum moved by more than half the offset. It reads one if it kept every value and none of those figures moved by a quarter of the offset. Every T2 run of both blocks was judged again under both rules, beside the verdict it was given. The corrected rule changes two verdicts, both for runs of Pulsar's client. The run above becomes undecided, and one undecided run becomes "drops the negatives".

Table S16 gives each tool's answer in its Clocks column. Twelve tools take both timestamps in one process, and moving the clock left their figures within a quarter of the offset in every one of their 96 runs. The other four timestamp a message in one process and subtract in another. The judge read librdkafka's `rdkafka_performance` as keeping every value in three of its four rounds, but its consumer drops any value at or below zero and prints a line for each. No such line appears in any of its runs, and none of its own values came near zero, so that verdict tested nothing. Pulsar's client drops a value below zero and keeps a zero. Under the larger offset it kept 3 to 7 of 3,000 latencies, yet its summary printed 3,000 records received beside latency figures computed from those few. The OpenMessaging Benchmark drops every value at or below zero (S4.5). `emqtt-bench`'s histogram kept every value, its sum going below zero with the clock. Its console printed no latency in any of its offset runs, because it prints its once-a-second average only when a counter that adds the latencies above zero changes.

A third test, T3, ran each tool idle and against a background load that kept every core 88% busy, each with and without real-time priority on the process that subtracts. The plan fixed no rule for T3, so what follows describes the runs and judges nothing. The load lengthened the tail of every tool that prints a 99th percentile and barely moved its median (Table S17). Taking the median over the four rounds, the 99th percentile of each of the 14 rose by 1.4–5.0 ms, of the order of the 2.8 ms slice a thread can wait for a core (Section IV-C), while the median moved by at most 0.2 ms. Real-time priority took at least nine tenths of the rise back in eleven of them. A tool times the wait for a core together with everything else it times, so the wait reads as latency.

The per-run verdicts and figures are in `docs/results/tools/tools_t1.csv`, `tools_t2.csv` and `tools_t3.csv`, folded by `scripts/tools_block.py` from the runs. The write-ups are `docs/results/tools/t1-t4-eleven-tools.md`, with its five recorded faults, and `docs/results/tools/t1-t4-five-tools.md`.

Public reports. Published results rarely carry the deletion's signature, and no report we found says how many samples it kept. The audit behind that sentence was registered, committed and public before any report was read (`freezes/29-published-results-audit`). It fixed the signature, the places to search, the rule for inclusion, the coding and the outcomes, and nothing in it changed afterwards.

The primary signature is a printed end-to-end median of exactly 1 ms, the smallest value the benchmark can print and the value every retained sample takes when the true end-to-end latency is shorter than the resolution. The strong signature adds a p99 of exactly 1 ms. Neither says a published number is wrong. Each says the number came from a fraction of the samples, a fraction the report does not state.

The search covered Google Scholar, Semantic Scholar, the web, the engineering blogs of the vendors whose brokers the benchmark has drivers for, and the benchmark's repository and its public copies, for reports dated up to 28 September 2026. A report was included if it named the benchmark or a fork of it as its tool and printed at least one numeric end-to-end percentile.

The audit included 43 reports, printing 1,100 configurations, of which 868 print a median. The primary signature occurs in eight configurations in two reports, both public repository text. Five are Redis rates in a notebook behind a 2023 journal paper that shows its medians only in a chart, and three are maximum-rate runs in three result files of a 2021 fork. The strong signature occurs nowhere, and no included report states its retention or a discard count.

The rules did not foresee several patterns. 9 of the included reports print medians that are not whole milliseconds, and two of them print medians below a millisecond, which the original benchmark's clock cannot produce. They come from copies whose maintainers changed how the span is timed, and for Kafka and Pulsar those copies still take the publish timestamp in

TABLE S16
TWELVE OF THE TOOLS RUN TIME ON ONE CLOCK, AND FOUR ON TWO. THE REGISTERED CAMPAIGN'S TOOLS BLOCK, FOUR ROUNDS PER TOOL ON ONE PAIR OF MACHINES, ELEVEN TOOLS IN SEPTEMBER AND FIVE IN OCTOBER. INTERVAL: WHAT THE TOOL TIMES, AS ITS INVOCATION WAS READ BEFORE ANY RUN. SLOPE: HOW FAR ITS FIGURE MOVED PER MILLISECOND OF DELAY, DIVIDED BY THE NUMBER OF TIMES THE INTERVAL CROSSES THE DELAY, IN ITS LOWEST AND HIGHEST ROUND. CLOCKS: HOW MANY CLOCKS ITS TWO TIMESTAMPS COME FROM, ANSWERED FROM THE OFFSET ALONE AS VERSION 34 ASKS. TWO MEANS THAT MOVING BACK THE CLOCK OF THE PROCESS THAT SUBTRACTS CUT WHAT THE TOOL KEPT, OR MOVED ITS AVERAGE, MEDIAN OR MINIMUM BY MORE THAN HALF THE OFFSET. ONE MEANS THAT IT KEPT EVERY VALUE AND NONE OF THOSE MOVED BY A QUARTER OF THE OFFSET. RUNS: THE OFFSET RUNS SO ANSWERED, OF ALL OF THAT TOOL'S. THE REST WERE UNDECIDED.

Tool	Interval	Slope	Clocks	Runs
hey	round trip	1.00–1.04	one	8/8
k6	round trip	1.00–1.02	one	8/8
memtier	round trip	0.96–1.00	one	8/8
NATS bench	round trip	1.00–1.03	one	8/8
Valkey benchmark	round trip	0.98–1.05	one	8/8
Vegeta	round trip	0.99–1.00	one	8/8
wrk2	round trip	0.92–1.04	one	8/8
YCSB	round trip	1.00–1.04	one	8/8
Kafka producer	to ack.	1.00–1.14	one	8/8
Kafka end-to-end	one process	1.01–1.02	one	8/8
NATS latency	one process	0.99–1.06	one	8/8
RabbitMQ PerfTest	one process	0.97–1.00	one	8/8
emqtt-bench	two processes	0.90–1.05	two	8/8
OpenMessaging Benchmark	two processes	0.56–1.04	two	5/8
Pulsar perf.	two processes	1.03–1.14	two	8/8
librdkafka perf.	two processes	0.96–1.05	two	8/8

TABLE S17
ON A BUSY MACHINE EVERY TOOL'S TAIL LENGTHENED AND ITS MEDIAN BARELY MOVED. T3: EACH TOOL IDLE AND AGAINST A BACKGROUND LOAD KEEPING EVERY CORE 88% BUSY; REAL-TIME: THE PROCESS THAT SUBTRACTS RAN AHEAD OF THE LOAD AT REAL-TIME PRIORITY. EACH FIGURE IS THE MEDIAN OVER THE FOUR ROUNDS, IN MILLISECONDS, FOR THE TOOLS THAT PRINT A 99TH PERCENTILE.

	Median		99th percentile		
Tool	idle	88%	idle	88%	88%, real-time
Kafka end-to-end	1.00	1.00	2.00	7.00	2.00
Kafka producer	1.00	1.00	2.00	5.50	2.00
hey	0.80	0.80	2.00	3.20	2.00
k6	0.72	0.72	0.85	2.95	0.85
memtier	0.69	0.66	0.82	3.22	0.74
NATS bench	1.31	1.33	1.39	4.22	3.94
NATS latency	0.67	0.68	0.76	3.21	0.74
OpenMessaging Benchmark	2.00	2.00	2.00	6.00	5.00
Pulsar perf.	3.00	3.00	4.00	8.00	6.00
RabbitMQ PerfTest	0.85	0.84	1.02	3.81	1.02
Valkey benchmark	0.51	0.53	0.54	2.41	0.56
Vegeta	0.76	0.76	1.60	3.52	1.56
wrk2	1.38	1.45	1.93	4.44	1.71
YCSB	0.73	0.63	0.86	2.93	0.77

whole milliseconds and keep the positivity filter. Five more reports only look finer: the benchmark's histogram prints a whole millisecond above about 262 ms as the top of a bucket wider than a microsecond, such as 507.001 ms, and their copies time the span as the original does. One 2024 fork prints p95, p99 and p999 all at exactly 1 ms beside an average just above it and no median column, so by the registered rule it does not count. Four configurations print a median of zero, from empty histograms. The coding, the search log and every excluded report with its reason are in `docs/results/external/published_results_audit` in the artifact (postmortem, S25.1).

One more pattern was read after the audit and is not one of its outcomes. The benchmark's RabbitMQ driver takes the publish timestamp from the message's AMQP timestamp property, which carries whole seconds, so every RabbitMQ end-to-end value it prints carries an extra zero to one second, set by where in its second the message was published. Two included reports print RabbitMQ percentiles spread evenly over a second, with medians near 500 ms and 99th percentiles near 990 ms, one at every rate and the other at light loads. That is the signature of the whole-second timestamp, and the registered signature, a median of 1 ms, could not see it. A vendor's copy fixed the timestamp in 2020 and did not return the fix to the original.

S6. THE REGISTERED LAW CAMPAIGN

A law for the negative-span rate, fixed before its runs, had one prediction confirmed on every campaign that tested it, three confirmed on one broker only, and the rest not confirmed (Table S18). The one confirmed everywhere is that load raises the rate at the shortest end-to-end latencies. Neither prediction that the rate falls, as the end-to-end latency grows, where a slice set by

hand puts it was confirmed, the predictions of how quickly it falls held on one broker at most, and Python's rate exceeded the registered limit relative to Java's. Frozen programs judged every prediction over 8,102 runs on machines the earlier campaigns never ran on.

S6.1. The law, and the design that tested it

The registered law says that the scheduler's settings alone fix where the negative-span rate falls as the end-to-end latency lengthens, and how wide the fall is. It assumes that a thread kept waiting waits out a whole slice s of another task, and then up to one tick h. The rate then sits on a plateau $c + p$ while the end-to-end latency D is shorter than the slice, falls across one tick, and rests on a floor c beyond it:

$$r(D) = c + p \cdot \begin{cases} 1, & D \le s, \\ (s + h - D)/h, & s < D < s + h, \\ 0, & D \ge s + h. \end{cases} \tag{S2}$$

Here p is the waiting probability of Equation 3, and c is the floor that ordinary short delays leave. The slice and the tick predict where the fall starts and how wide it is; the plateau's height and the floor are measured. In the plan, a reading is negative when the thread that reads the clock for the acknowledgment waits longer than the margin $D - g$, a run's median end-to-end latency less its median publish latency g, the plan's letter for A. The campaign lengthens only the path to the consumer, so within a setup g stays put and the margin moves one for one with D.

Three pairs of Microsoft Azure machines in one region ran the campaign, each a driver with eight virtual CPUs and a broker with two; two pairs have AMD EPYC 9V74 processors and one has Arm Neoverse N2. The plan's blocks A1 to A9 are written L1 to L9 here. L1 sets the slice on the first x86 pair, and L4 on the Arm pair. L2 sets the tick, on three kernels built from one source tree that differ in the tick alone: 1, 4 and 10 ms. L3 sets the load, L5 the number of online CPUs, L7 real-time priority for the threads that read the clock, and L8 the client, Kafka's own Java client against our Python one. L6 records every `python3` thread's waits in half of L1's and L3's runs, and L9 records each of those threads' scheduling events in half its own runs. Each block carries numbered predictions, stated and judged in Table S18. A further block, M0, asked a question about bursty traffic that bears on none of the claims here.

Each run replays messages at a steady 50 a second and measures its end-to-end latency, publish latency and negative-span rate after a 30 s warm-up. The end-to-end latency is set by delaying only the traffic to the receiver's own network address, so the acknowledgment's path is untouched. Each session first calibrates how far each added delay moves the end-to-end latency, and an end-to-end latency shorter than the client's own with nothing added cannot be set, and was not run. The integrity rule judges every run as it ends. A run counts when every condition it was meant to have took effect, from the load within 3 points of its target to the added delay within $50\,\mu$s of its setting. Otherwise it is repeated, up to three attempts. A stopping rule on the publish latency stopped a campaign whose median publish latency moved too far, until the final version made it only record. The plan's own name for the publish latency, and for the stopping rule on it, is "got-it".

A round runs every setup once. Each campaign that tests a prediction took its rounds, at least four and at most forty, from simulating its own decision rule on data that follow the law. L2 and L9, which have no rule the simulation can take, ran at the floor of four, and no campaign ran more than 24. A prediction about where the fall sits is read twice, on a free reading, a curve required only never to rise, and on the law's fitted shape, and it passes only where both agree. The judging read 8,102 runs in 156 campaigns, taken from 16 to 27 September 2026, and 7,996 of them counted.

The plan was first frozen as version 5 on 16 September 2026, before its first run, and the version in force at the end is the thirty-second. A frozen version is never edited. A change is a new, numbered freeze, public before the runs it covers, whose first section says what changed, why, and whether a result already seen prompted it. Three kinds of change followed the campaigns' own results. Version 14, after L3's first campaign could not place the fall at 50% load, judged P3a only where the plateau reaches 2%. The stopping rule was corrected five times, each time after it had stopped a campaign. And three versions corrected judging programs that had already judged, each stating what the verdicts it touches had been. Blocks added as the campaign ran, in versions 24 and 27 to 29, were each read by a program fixed before their runs, and P6 was given its program in version 30, at a margin no run had been read against. Every freeze is in the repository's `freezes/` folder, whose index says what each one changed and whether a result prompted it.

S6.2. Load raised the rate everywhere, and the rate did not fall where the law put it

Load raised the rate at the shortest end-to-end latencies in all five load campaigns, all on Kafka, with every rise's interval above zero. Table S18 gives each verdict as its frozen program returned it. On Kafka, real-time priority took the plateau to zero in every run, and the fall followed the core count at the one count whose slice is at least Kafka's shortest end-to-end latency. Kafka's own Java client shows the same fall and the same effect of priority, at a plateau a third to a quarter of the Python client's. The Arm pair has values of S below zero as the x86 pairs do (Figure S3).

On the first x86 pair the free reading placed the fall's halfway point inside its band at every slice each broker reaches, four for Kafka and five for Redis. The fitted shape did so at two of four slices for Kafka and two of five for Redis. On the Arm

pair the free reading held on both brokers and the fitted shape failed on both. P1 and P9 need both readings, so neither is confirmed. Load moved the fall toward shorter end-to-end latencies in every campaign, and P3a held in none.

TABLE S18
ONE PREDICTION CONFIRMED ON EVERY CAMPAIGN, THREE ON ONE BROKER, AND THE REST NOT. EVERY PREDICTION REGISTERED FOR BLOCKS L1 TO L9, THE NUMBER IT WAS JUDGED ON (ALL RUNS; A SLOPE, RATIO OR MOVE WITH ITS 95% INTERVAL, FROM RESAMPLING WHOLE ROUNDS), AND ITS VERDICT, BROKER BY BROKER WHERE THE TWO DIFFER. A PREDICTION JUDGED BROKER BY BROKER IS CONFIRMED ONLY WHERE IT HOLDS ON BOTH.

	Block, pair	The prediction	On all runs	Verdict
P1	L1, first x86	the fall's halfway point inside $[s, s+h]$ at all but one of the slices a pair reaches, at least four of them; its slope on s in 0.8–1.2	slope, free and fitted: Kafka 0.758 (0.596–1.023) and 0.230; Redis 1.007 (0.844–1.169) and 0.662	not confirmed
P9	L4, Arm	on the Arm pair, inside at every slice it reaches, at least two; the same slope	Kafka 0.830 (0.627–1.172) and 0.285; Redis 1.153 (0.753–1.325) and 0.798	not confirmed
P2	L2	the fall 2.5 to 5.5 times wider at a 4 ms tick than at 1 ms	ratio, free and fitted: Kafka 1.784 and 1.818; Redis 1.064 and 1.126 (S6.3)	not confirmed
P2b	L2	6 to 14 times wider at a 10 ms tick	Kafka 4.786 and 5.358; Redis 3.045 and 3.339 (S6.3)	not confirmed
P2c	L2	the width on the tick a line through zero, slope 0.8–1.2	fitted slope: Kafka 0.968 (0.271–1.125); Redis 0.874 (0.807–1.034)	confirmed on Redis only
P2d	L2	the fitted start within 0.25 ms of the slice at every tick	within it at one of twelve slice and tick pairs	not confirmed
P3a	L3, five Kafka campaigns	the halfway point moves less than 0.25 ms from 75% to 88% load	free move: −0.009, −0.188, −0.086, −0.011 and −0.309 ms	not confirmed on any
P3b	L3, five Kafka campaigns	load raises the plateau	rise, free: 0.048 (0.044–0.052), 0.047, 0.048, 0.041 and 0.050	confirmed on all five
P4	L7, first x86	real-time priority cuts the plateau at least five times wherever it is at least 2%	Kafka 0.0405 to zero; Redis's plateau 0.0069 (S6.3)	confirmed on Kafka only
P6	L6	a recording of every thread's waits predicts the rate within 0.67–1.5	median ratio 2.49 to 13.08 over four pair and broker parts; no setup inside	not confirmed
P7	L5, second x86	the default slice follows the core count, and the fall follows it	slices read back at 2, 4 and 8 CPUs as the rule gives; the fall on Kafka at 8 CPUs, Redis's fitted shape missing at all three	confirmed on Kafka only
P8	L8, first x86 and Arm	Java has the same fall and priority effect, and Python's plateau is at most 1.5 times Java's	Python over Java: 3.251 (2.795–3.907), and 4.253 (4.013–4.560) on Arm	not confirmed
L9-1	L9, three pairs	at least 90% of negative readings mostly spent waiting for a CPU	79.4 to 92.0%	not confirmed
L9-2	L9	the time-weighted waits predict the rate within 0.67–1.5	median ratio 0.104 to 0.165	not confirmed
L9-2b	L9	the share of acknowledgments whose own wait outlasted $D - g$ predicts it	inside the rule on four of six parts; median ratio 1.11 to 1.30	not confirmed

The plan keeps every run, and re-runs none whose conditions held. Void runs, runs the integrity rule did not count, and the 18 runs of six false starts, each false start replaced at once by a campaign of the same design, are judged by no program. Four marks leave a run counted: flagged, paused, far from its setup's other repeats, and traced. The stopping rule would have stopped the 14 flagged runs, taken while it only recorded. In the 222 paused runs a message arrived more than 150 ms late, either because the receiving side's disk stalled, for a cause not identified, or because Kafka's broker could not reach its own controller. No run had a message arrive before it was published. Without the flagged runs, or without all 1,110 taken while the stopping rule only recorded, no verdict moves. Without the paused runs every verdict stands but P3a on the first x86 pair, whose fitted interval sits on the edge of its ± 0.25 ms bar, just outside it with them and just inside without. The verdict on all the runs is the one reported. The verdicts, on all runs and on each subset, are under `docs/results/law/judged-27-sep`.

S6.3. Read after the verdicts: the wait is the rest of a slice, and two judging programs departed from the plan

Everything here was read after the verdicts, and none of it changes one. The rate holds no plateau. At half the slice it is 1.45 to 3.94 times its value at 0.9 of the slice, in each of the ten slice and broker combinations whose runs reach half the slice.

L9's recordings, at a 3 ms slice and a 1 ms tick, time the wait of the thread that reads the clock for each acknowledgment, for 1,516,342 acknowledgments in 304 runs. Of those whose thread waited more than 1 ms, 24.3 to 35.8% were still waiting at 3 ms and 3.3 to 6.8% past 4 ms, over twelve pair, broker and load parts. A thread preempted for a task starting its slice waits out all of it, but a thread woken while another task holds the core waits out what is left. The acknowledgment's reply wakes the thread that reads it, so its wait is the second kind. A whole-slice wait would leave all of them still waiting at 3 ms; a

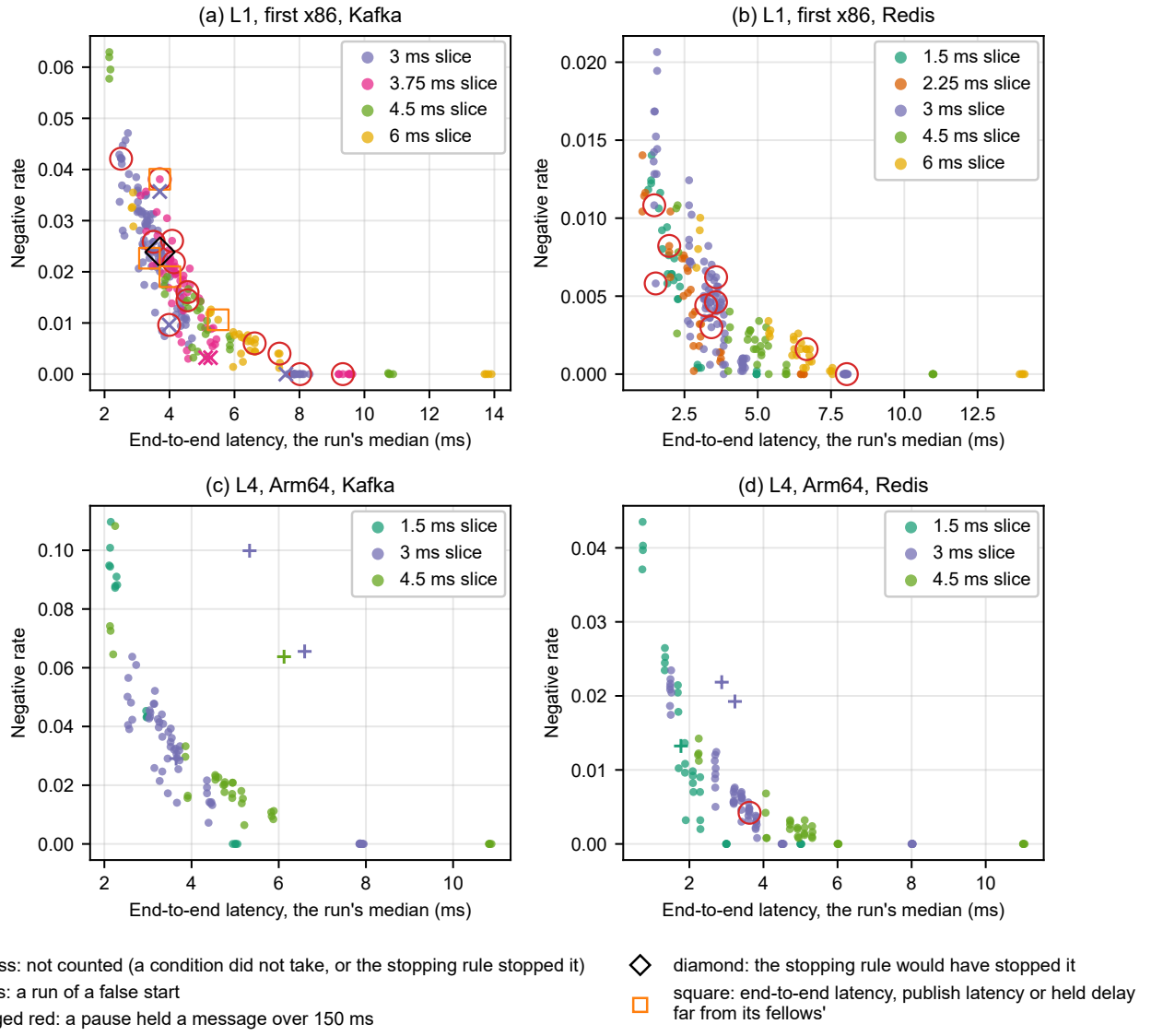


Fig. S3. **Slice runs fall from the shortest end-to-end latency on, where the registered law put a plateau.** Every run of L1 (first x86 pair) and L4 (Arm pair, Arm64 in the panels) at its median end-to-end latency and its negative-span rate, colored by the slice it ran at. Crosses: runs the integrity rule did not count. Plusses: runs of L4's two false starts. Rings, diamonds and squares: counted runs that are paused, flagged, or far from their setup's other repeats. That the rate holds no plateau was read after the verdicts (S6.3).

wait for the rest of the slice leaves 20% waiting at 3 ms and none past 4 ms. That thread waits out what is left of the running task's slice, R, uniform on $[0, s]$, and then up to a tick, U, uniform on $[0, h]$, so that for $h \le s$

$$G(x) = \Pr[R + U > x] = \begin{cases} 1 - x^2/(2sh), & 0 \le x \le h, \\ 1 - (x - h/2)/s, & h \le x \le s, \\ (s + h - x)^2/(2sh), & s \le x \le s + h, \\ 0, & x \ge s + h. \end{cases} \tag{S3}$$

The recording raised the rate it recorded, 1.36 to 1.49 times for Kafka and 3.69 to 4.33 times for Redis, so these waits describe the machines with it on. Fitted to a rate that falls from the shortest end-to-end latency on, the law's shape puts its fall below the slice and moves it less than the slice moves, which is why the free and fitted readings of P1 and P9 split.

In two places a frozen program departed from the plan, and in both the frozen answer is the one reported. The programs judging P2, P2b and P2c group L2's runs by tick alone, so each width they read spans the falls of both its slices, 1.5 and 3 ms. With Equation S2 in place of every run's rate, as if the law held exactly, the P2 program returns 1.98 free and 1.57 fitted for Redis, below its band. One slice at a time, the same formula gives 4.0 to 5.2 for P2 and 9.8 to 12.0 for P2b, inside both bands. Those three verdicts therefore carry no evidence about the law either way. P4's program words Redis "not confirmed"

although its plateau, 0.69%, lies below the 2% at which P4 makes a claim, and the plan's rule reports a plateau that low as out of reach.

REFERENCES

[S1] StatsBomb, "Statsbomb open data," *GitHub Repository*, 2023. [Online]. Available: https://github.com/statsbomb/open-data

[S2] D. Katz, J. Baptista, S. P. Azen, and M. C. Pike, "Obtaining confidence intervals for the risk ratio in cohort studies," *Biometrics*, vol. 34, no. 3, pp. 469–474, 1978.

[S3] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *J. Amer. Statist. Assoc.*, vol. 22, no. 158, pp. 209–212, 1927.

[S4] Linux Kernel Documentation, "KVM-specific MSRs: `MSR_KVM_SYSTEM_TIME_NEW`," https://docs.kernel.org/virt/kvm/x86/msr.html, 2026, accessed: Aug. 21, 2026.

[S5] Z. Zhou, "sched: Reduce the default slice to avoid tasks getting an extra tick," Linux kernel mailing list, commit 2ae891b82695, 2025.

[S6] Hewlett-Packard, "Time interval averaging," Hewlett-Packard Company, Application Note 162-1, 1970, part no. 02-5952-0766.

[S7] Apache Software Foundation, "Apache Kafka tools — `EndToEndLatency.java` and `ProducerPerformance.java`," Source repository, 2026, https://github.com/apache/kafka. Accessed: Aug. 31, 2026.

[S8] Apache Pulsar contributors, "`PerformanceConsumer.java`," Source repository, 2026, https://github.com/apache/pulsar. Accessed: Aug. 21, 2026.

[S9] Synadia, "NATS cli — `cli/latency_command.go`," Source repository, 2026, https://github.com/nats-io/natscli. Accessed: Oct. 2, 2026.

[S10] OpenMessaging Project, Linux Foundation, "The OpenMessaging benchmark framework," 2018, available at openmessaging.cloud/docs/benchmarks/. Accessed: Aug. 19, 2026.

[S11] Broadcom, "RabbitMQ PerfTest — `TimestampProvider.java`," Source repository, 2026, https://github.com/rabbitmq/rabbitmq-perf-test. Accessed: Oct. 2, 2026.

[S12] IOP Systems, "Rezolus: systems performance telemetry — `scheduler_discarded_samples`," Source repository, 2026, https://github.com/iopsystems/rezolus. Accessed: Aug. 20, 2026.

[S13] A. D. Brunelle, "`blktrace`/`btt` — `btt/inlines.h`," Source repository, 2026, accessed: Aug. 20, 2026.

[S14] EMQ Technologies, "`emqtt-bench`: MQTT benchmark tool — `src/emqtt_bench.erl`," Source repository, 2026, https://github.com/emqx/emqtt-bench. Accessed: Aug. 21, 2026.

[S15] J. Axboe, "fio: Flexible I/O tester — `gettime.c`," Source repository, 2026, https://github.com/axboe/fio. Accessed: Aug. 20, 2026.

[S16] Confluent, "`librdkafka` — `examples/rdkafka_performance.c`," Source repository, 2026, https://github.com/confluentinc/librdkafka. Accessed: Oct. 11, 2026.

[S17] G. Tene, "`wrk2` — `src/wrk.c`," Source repository, 2026, https://github.com/giltene/wrk2. Accessed: Aug. 21, 2026.